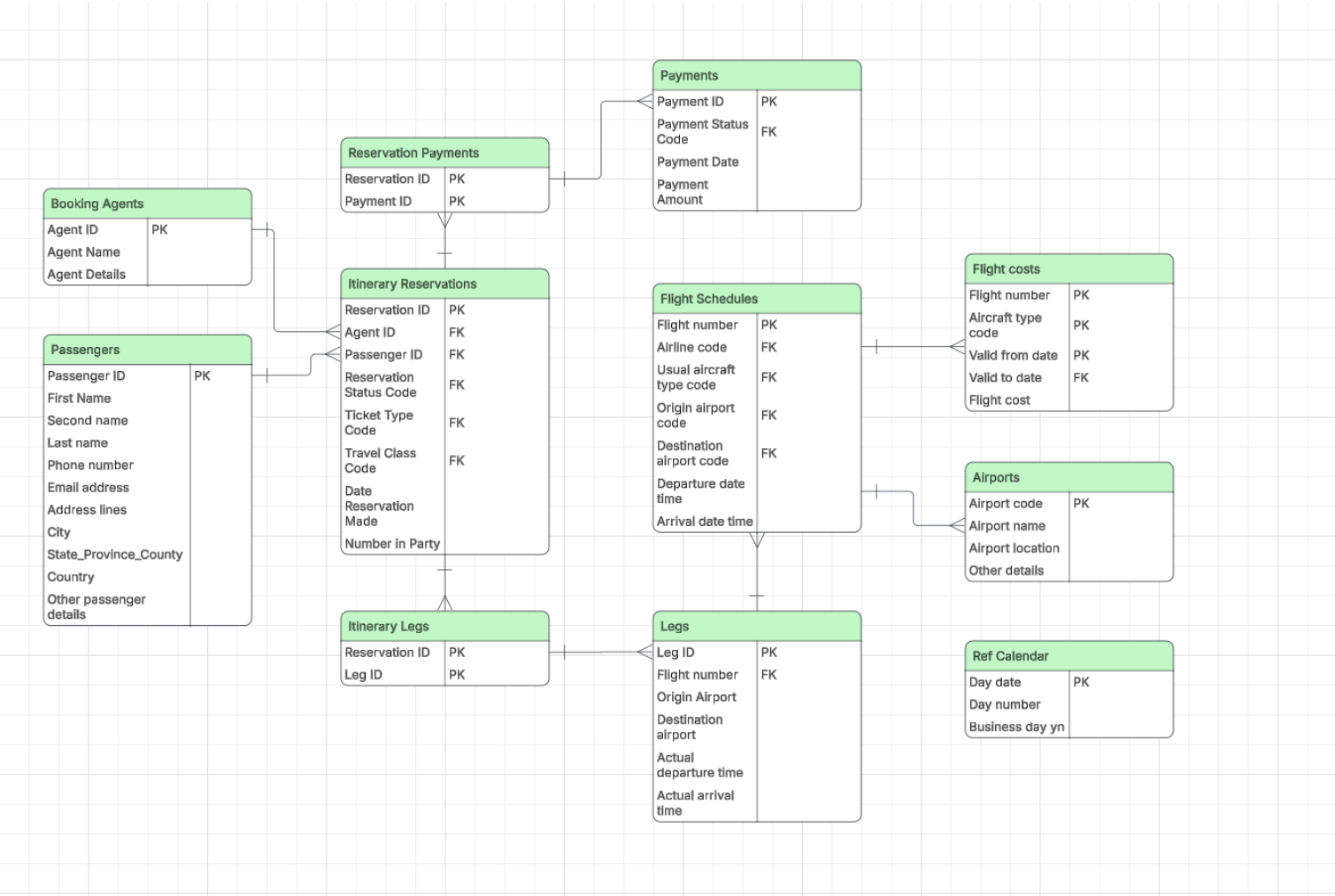
Chase Golden

Project 01 – SQL Project

Part 1 – Data Model (ERD)



Part 2 – Implementation

- **Write the DDL (CREATE) statements to create the database implementing the PK and FK constraints.**

```
-- Airports
CREATE TABLE Airports (
  Airport_Code CHAR(3) PRIMARY KEY,
  Airport_Name VARCHAR(100) NOT NULL,
  Airport_Location VARCHAR(100),
  Other_Details TEXT
);

-- Booking Agents
CREATE TABLE Booking_Agents (
  Agent_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Agent_Name VARCHAR(100) NOT NULL,
  Agent_Details TEXT
);

-- Passengers
CREATE TABLE Passengers (
  Passenger_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  First_Name VARCHAR(50) NOT NULL,
  Second_Name VARCHAR(50),
  Last_Name VARCHAR(50) NOT NULL,
  Phone_Number VARCHAR(20),
  Email_Address VARCHAR(100),
  Address_Lines TEXT,
  City VARCHAR(50),
  State_Province_County VARCHAR(50),
  Country VARCHAR(50),
  Other_Passenger_Details TEXT
  -- Optional: UNIQUE (Email_Address)
);
```

```sql
-- Flight Schedules
CREATE TABLE Flight_Schedules (
  Flight_Number VARCHAR(12) PRIMARY KEY,
  Airline_Code VARCHAR(10) NOT NULL,
  Usual_Aircraft_Type_Code VARCHAR(20) NOT NULL,
  Origin_Airport_Code CHAR(3) NOT NULL REFERENCES Airports(Airport_Code),
  Destination_Airport_Code CHAR(3) NOT NULL REFERENCES Airports(Airport_Code),
  Departure_Date_Time TIMESTAMP NOT NULL,
  Arrival_Date_Time TIMESTAMP NOT NULL,
  CHECK (Origin_Airport_Code <> Destination_Airport_Code)
);

-- Legs
CREATE TABLE Legs (
  Leg_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Flight_Number VARCHAR(12) NOT NULL REFERENCES Flight_Schedules(Flight_Number) ON DELETE CASCADE,
  Origin_Airport CHAR(3) NOT NULL REFERENCES Airports(Airport_Code),
  Destination_Airport CHAR(3) NOT NULL REFERENCES Airports(Airport_Code),
  Actual_Departure_Time TIMESTAMP,
  Actual_Arrival_Time TIMESTAMP,
  CHECK (Origin_Airport <> Destination_Airport)
);

-- Itinerary Reservations
CREATE TABLE Itinerary_Reservations (
  Reservation_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Agent_ID INT REFERENCES Booking_Agents(Agent_ID),      -- NULL = direct booking allowed
  Passenger_ID INT NOT NULL REFERENCES Passengers(Passenger_ID),
  Reservation_Status_Code VARCHAR(12) NOT NULL,
  Ticket_Type_Code VARCHAR(12) NOT NULL,
  Travel_Class_Code VARCHAR(12) NOT NULL,
  Date_Reservation_Made DATE NOT NULL,
  Number_in_Party INT NOT NULL CHECK (Number_in_Party > 0)
);
```

```
-- Itinerary Legs
CREATE TABLE Itinerary_Legs (
  Reservation_ID INT NOT NULL REFERENCES Itinerary_Reservations(Reservation_ID) ON DELETE CASCADE,
  Leg_ID INT NOT NULL REFERENCES Legs(Leg_ID) ON DELETE CASCADE,
  PRIMARY KEY (Reservation_ID, Leg_ID)
);

-- Payments
CREATE TABLE Payments (
  Payment_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Payment_Status_Code VARCHAR(12) NOT NULL,
  Payment_Date DATE NOT NULL,
  Payment_Amount NUMERIC(10,2) NOT NULL CHECK (Payment_Amount >= 0)
);

-- Reservation Payments
CREATE TABLE Reservation_Payments (
  Reservation_ID INT NOT NULL REFERENCES Itinerary_Reservations(Reservation_ID) ON DELETE CASCADE,
  Payment_ID INT NOT NULL REFERENCES Payments(Payment_ID) ON DELETE CASCADE,
  PRIMARY KEY (Reservation_ID, Payment_ID)
);

-- Flight Costs
CREATE TABLE Flight_Costs (
  Flight_Number VARCHAR(12) NOT NULL REFERENCES Flight_Schedules(Flight_Number) ON DELETE CASCADE,
  Aircraft_Type_Code VARCHAR(20) NOT NULL,
  Valid_From_Date DATE NOT NULL,
  Valid_To_Date DATE,
  Flight_Cost NUMERIC(10,2) NOT NULL CHECK (Flight_Cost >= 0),
  PRIMARY KEY (Flight_Number, Aircraft_Type_Code, Valid_From_Date)
);

-- Ref Calendar
```

```
CREATE TABLE Ref_Calendar (
  Day_Date DATE PRIMARY KEY,
  Day_Number INT NOT NULL CHECK (Day_Number BETWEEN 1 AND 366),
  Business_Day_YN CHAR(1) NOT NULL CHECK (Business_Day_YN IN ('Y','N'))
);
```

- **Write the SQL queries/views for the highlighted requirements in section 2.3**

```
--View a passenger's complete itinerary
CREATE OR REPLACE VIEW v_passenger_itinerary AS
SELECT
    p.passenger_id,
    p.first_name,
    p.last_name,
    ir.reservation_id,
    fs.flight_number,
    a1.airport_name AS origin_airport,
    a2.airport_name AS destination_airport,
    fs.departure_date_time,
    fs.arrival_date_time
FROM Passengers p
JOIN Itinerary_Reservations ir ON ir.passenger_id = p.passenger_id
JOIN Itinerary_Legs il          ON il.reservation_id = ir.reservation_id
JOIN Legs l                     ON l.leg_id = il.leg_id
JOIN Flight_Schedules fs        ON fs.flight_number = l.flight_number
JOIN Airports a1 ON a1.airport_code = fs.origin_airport_code
JOIN Airports a2 ON a2.airport_code = fs.destination_airport_code;

--1. Get all customers who have seats reserved on a given flight
CREATE OR REPLACE VIEW v_customers_on_flight AS
SELECT DISTINCT
    fs.flight_number,
    p.passenger_id,
```

```
    p.first_name,
    p.last_name,
    ir.reservation_id
FROM Passengers p
JOIN Itinerary_Reservations ir ON ir.passenger_id = p.passenger_id
JOIN Itinerary_Legs il          ON il.reservation_id = ir.reservation_id
JOIN Legs l                     ON l.leg_id = il.leg_id
JOIN Flight_Schedules fs        ON fs.flight_number = l.flight_number;

--2. Get all flights for a given airport
CREATE OR REPLACE VIEW v_flights_for_airport AS
SELECT
    fs.flight_number,
    fs.origin_airport_code,
    fs.destination_airport_code,
    fs.departure_date_time,
    fs.arrival_date_time
FROM Flight_Schedules fs;

--3. View flight schedule
CREATE OR REPLACE VIEW v_flight_schedule_named AS
SELECT
    fs.flight_number,
    a1.airport_name AS origin_airport,
    a2.airport_name AS destination_airport,
    fs.departure_date_time,
    fs.arrival_date_time
FROM Flight_Schedules fs
JOIN Airports a1 ON a1.airport_code = fs.origin_airport_code
JOIN Airports a2 ON a2.airport_code = fs.destination_airport_code;

--4. Get all flights whose arrival and departure times are on time/delayed
CREATE OR REPLACE VIEW v_leg_punctuality AS
SELECT
```

```
    l.leg_id,
    l.flight_number,
    CASE
        WHEN l.actual_departure_time = fs.departure_date_time
         AND l.actual_arrival_time   = fs.arrival_date_time THEN 'ON_TIME'
        WHEN l.actual_departure_time IS NULL OR l.actual_arrival_time IS NULL THEN 'UNKNOWN'
        ELSE 'DELAYED'
    END AS status
FROM Legs l
JOIN Flight_Schedules fs ON fs.flight_number = l.flight_number;

--5. Calculate total sales for a given flight
CREATE OR REPLACE VIEW v_sales_by_flight AS
SELECT
    l.flight_number,
    SUM(p.payment_amount) AS total_sales
FROM Payments p
JOIN Reservation_Payments rp   ON rp.payment_id = p.payment_id
JOIN Itinerary_Reservations ir ON ir.reservation_id = rp.reservation_id
JOIN Itinerary_Legs il         ON il.reservation_id = ir.reservation_id
JOIN Legs l                    ON l.leg_id = il.leg_id
GROUP BY l.flight_number;
```

Part 3 – Discussion

1. What types of performance constraints do you see Customers potentially facing if you do not create Views for the queries described in the CUSTOMER FUNCTIONS section?

If views weren't created to assist the customer functions in Section 2.3 of the SRS, the customers would likely endure slower response times and poorer performance. In a relational database, like this airline reservation system, a customer request such as "view my itinerary" or "list flights for a given airport" must gather data from several tables that work together: Passengers, Itinerary Reservations, Itinerary Legs, Legs, and Flight Schedules. If views did not exist, each instance of the query will need to reperform all of the complex JOINS for every iteration of the query. This process requires a lot of additional processing time and memory and can be burdensome on the underlying database.

Without views, when a user issues one of these problems, the underlying database must compile and execute the same complex logic every time. This causes redundancy, which can lead to varying performance from the perspective of the user. In an operational airline system, there can be hundreds or thousands of customers checking the schedule, booking flights or checking payment status at the same time. If every request executes all the JOINS independently, you can imagine the latency in the system. Views function like a performance layer. A dozen views may store the optimized joins under the hood so that the database can run the data requests faster, and consistently lower time to load a page, and a reasonable cost of computation and data retrieval. Lack of views will lead customers to see longer loading times, delayed updates to flights and payment, and overall, a less responsive booking capability.

2. Do you think this creates more overhead for the DB?

I do think that not using views requires more overhead for both developers and the database engine. From the developer's perspective, each function in the app that retrieves either flight data or itinerary data would have a long SQL query, including multiple JOINS, making the application potentially unmaintainable or error prone. If there is any structural change to a single table, whether a column has been renamed or a constraint has been modified, you would need to change each of those queries across the entire application. Views centralize that logic, so that any schema change is cognized in one place and is reflected throughout the application.

From the database's perspective, overhead is added because no optimization can occur when the same query is run over and over again as both plain SQL and then described in the application code. When there is a new SQL query received from the application, the database must parse, plan, and execute it from a new state instead of potentially reusing an execution plan. This process involves unnecessary utilization processing and memory resources. When hundreds of users have the same kind of queries, the database simply recreates the same workload and inefficiently executes and keeps generating lock contention. A view reduces the overhead because it can optimize the CPU processing and memory work if you plan to run the same query multiple times concurrently.

In the absence of views, the system could end up performing either temporary computations or scans of the full table every time a user requested some data. These computations increase loads on storage and memory resources for no reason. Eventually, when both the volume of traffic and data increase, the repetitive requests will cause the system to become less responsive overall and less stable.

3. Do you think query performance will scale as the database scales up?

If views are not used, query performance will not scale well as the database grows. When the system is only holding a small amount of data, queries that join a few tables may run quickly. But if the volume of passengers, flights, reservations, and payment records climbs to the millions, the previous defined queries will take much longer to run. Multi-table joins are particularly sensitive to dataset size because as many tables are joined, the database must process and match that many more records.

As the system scales up, the database workload increases in a few different ways. First, storage and retrieval operations take longer due to larger indexes and data pages that must be traversed as the underlying dataset grows. Second, concurrency becomes a problem; as many users access the same tables concurrently, their resource usage will compete with one another. Without views or simple caching, the same complex JOINS will be re-calculated to serve each user session. Recalculating these items will slow throughput and cause bottlenecks.

Similarly, as the data volume increases, certain tasks like the total sales per flight, or rendering customer itineraries will proceed to consume more and more resources. Even with indexing and foreign keys, a growing number of relationships across tables will impact execution time consistently and severely. The price of maintaining scalability will be simple views, aggregations, and other optimizations to ensure queries are simplistic, and reuse

them, and predefine the relationships across key tables. If these did not exist, performance would diminish over time, and the database would struggle to maintain the user demand.

4. How would you alleviate any potential scaling problems with the DB?

There are a variety of performance-oriented strategies that can be implemented to combat scaling issues in the database for airlines. The first is indexing. Indexing allows the database to search for and join records more quickly. When creating indexes, it is important to consider which fields can be designated as keys. In the context of airlines, the fields of Passenger ID, Reservation ID, and Flight Number are ideal candidates for indexing, as they will reduce ultimately query times.

Caching is also an option to store results from data that is accessed frequently, such as flight schedules or passenger itineraries, to avoid reprocessing tasks, as the queries can be processed more quickly to memory instead of repeatedly to storage. Partitioning is a third option which entails creating a separate table for large tables; the flights table could be partitioned by year or region, which would cause the queries to scan only relevant partitions instead of the entire flights table dataset.

Finally, replication and sharding would involve moving to a cluster model to different servers, that would balance the workload and improve response times in larger environments. All of these strategies, along with thoughtful and well-designed views, are intended to allow the database to remain high performing, scalable, and reliable, regardless of the increased number of users in addition to transactions/activities.

5. Do you think a first-normal form RDBMS is best suited for a highly transactional web application?

A relational database that is in first-normal-form is a sound underpinning for a transactional web application because it foils structure, there is no data duplication, and it provides accuracy. Normalizing can jeopardize performance in certain cases, particularly when each transaction involves high-volume, constant transactions. For example, when a customer books or changes a flight, they will have to do multiple joins across several tables, which adds time and computational expense.

Many large applications typically use a hybrid approach. They normalize the core relational database to keep integrity but denormalize a few fields for the service of faster access. As an example, if I have a customer record with a Passengers table, I might store a customer's last booking data in the Passengers table for faster lookup. Implementing this kind of

approach by combining an RDBMS that is normalized for important transactional data with other types of systems for caching or NoSQL for bigger data analytics, allows the system to be both fast and accurate.

So, while a relational database that is in 1NF is good for consideration and accuracy, a transactional web application adds other performance considerations to a normalized relational database, when real-time and high-transactional activity is in place.

Sources:

Differences between SQL Tables, Views, and Materialized Views! | by Ljscodex | Medium

The Complete Guide to System Design in 2025

Top Tips for Creating Efficient SQL Views